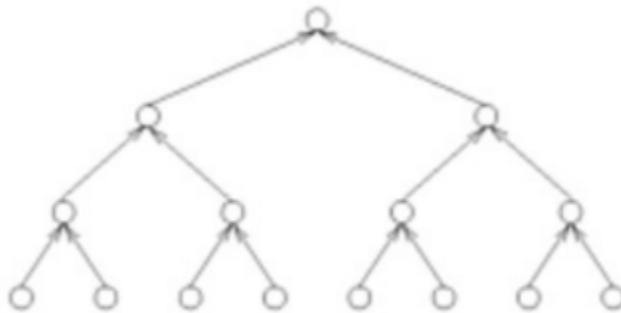


CS 5200 HW 2

Q1. See the dependency graphs on Slide #36 (Dynamic Task Parallel Algorithms ppt). Find the work, span and parallelism of each graph. (5 points = $1/2 + \frac{1}{2} + 2 + 2$)

Graph 1:



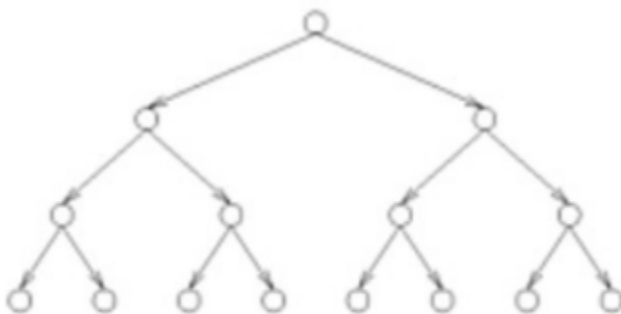
$$N=2^n - 1, n = 4 \quad (a)$$

Work: $8 + 4 + 2 + 1 = 15$

Span: $1+1+1+1=4$

Parallelism: $15/4 = 3.75$

Graph 2:



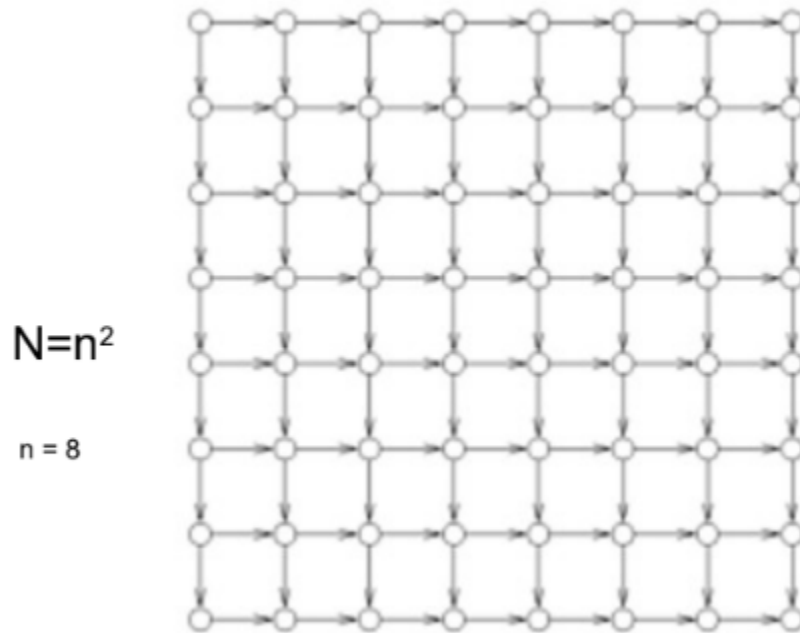
$$(b) \quad N=2^n - 1, n = 4$$

Work: $8 + 4 + 2 + 1 = 15$

Span: $1+1+1+1=4$

Parallelism: $15/4 = 3.75$

Graph 3:

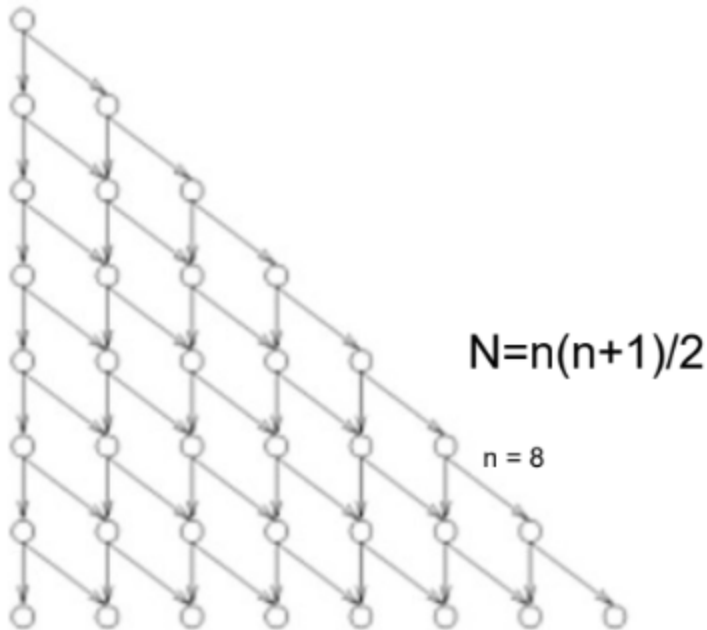


Work: $8 \times 8 = 64$

Span: $14 + 1 = 15$

Parallelism: $64/15 = 4.2666666667$

Graph 4:



Work: 36

Span: 8

Parallelism: $36/8 = 4.5$

Q2. Find out the parallelism of the following algorithms

```
void transpose1( int A[], int n)
{
    parallel for j = 2 to n
    for i = 1 to j - 1
    exchange A[i][j] = A[j][i]
}
```

The outer loop runs for $j = 2, 3, 4, \dots, n$

The inner loop runs $j - 1$ times

Total exchanges = $1 + 2 + 3 \dots + (n - 1) = \frac{n(n-1)}{2}$

Work: $O(n^2)$

The outer loop is parallel for. Spawning a parallel loop of size n using divide-and-conquer takes $\Theta(\log(n))$ overhead

The inner loop is a standard serial for loop. Its span is the time it takes to run its longest iteration.

The longest iteration happens when $j=n$, where the inner loop runs $n-1$ times. The span of this serial loop is $\theta(n)$

Total Span = (Overhead of outer parallel loop) + (Max span of the inner loop).

$$\text{Span: } T_{\infty} = O(\text{Log}(n)) + O(n) = O(n)$$

$$\text{Parallelism: } \frac{T_1}{T_{\infty}} = \frac{O(n^2)}{O(n)} = O(n)$$

```
void transpose2( int A[], int n )
{
    parallel for j = 2 to n
    parallel for i = 1 to j - 1
    exchange A[i][j] = A[j][i]
}
```

Parallelizing loops does not change the total amount of computational work done.

$$\text{Work: } O(n^2)$$

The outer loop is parallel for. Overhead = $\theta(\log n)$

The inner loop is also a parallel for. The overhead for the inner loop at its maximum size (when $j=n$) is also $\theta(\log n)$

The actual exchange instruction takes $\theta(1)$ time.

Total Span = (Overhead of outer loop) + (Max overhead of inner loop) + (Work of the exchange)

$$\text{Span: } T_{\infty} = O(\text{Log}(n)) + O(\text{Log}(n)) + O(1) = O(\text{Log}(n))$$

$$\text{Parallelism: } \frac{T_1}{T_{\infty}} = \frac{O(n^2)}{O(\text{Log}(n))}$$

Q3. In an unsorted array of numbers, count the number of pairs that are out of order. For example, in array {1, 20, 6, 4, 5}, pair (1,20) is in-order. However, pair (20,6) is not in order

If you have an unsorted array {1, 20, 6, 4, 5}, then out of order pair of numbers in the array are (20, 6), (20, 4), (20, 5), (6, 4), (6, 5).

Write a parallel algorithm to solve the counting problem. Find the work, span and parallelism of your algorithm assuming n integers in the array and unlimited number of processors.

```
int sort(A, low, high)
{
    // 0 or 1 element has 0 inversions
    if (low >= high) {
        return 0;
    }

    int mid = (low + high) / 2;

    // Recursively sort and count halves in parallel
    int left_inv = spawn sort(A, low, mid);
    int right_inv = sort(A, mid + 1, high);

    // Wait for the spawned left half to finish
    sync;

    // merge and count split inversions
    int split_inv = merge(A, low, mid, high);
    return left_inv + right_inv + split_inv;
}

// sequential merge
int merge(A, low, mid, high)
{
    // standard merge logic
    int i = low;
```

```

int j = mid + 1;
int k = 0;
int count = 0;
temp_array = new array of size (high - low + 1);

while (i <= mid AND j <= high) {
    if (A[i] <= A[j]) {
        temp_array[k++] = A[i++];
    } else {
        // Inversion
        // All remaining elements in the left half are also > A[j]
        temp_array[k++] = A[j++];
        count = count + (mid - i + 1);
    }
}

while (i <= mid) {
    temp[k++] = A[i++];
}

while (j <= high) {
    temp[k++] = A[j++];
}

// Copy temp sorted array back into original array A
for (int p = 0; p < k; p++) {
    A[low + p] = temp[p];
}

return count;
}

```

Since this is the same work as merge sort, the work will be $O(n \log(n))$. The two recursive calls (left_inv and right_inv) run in parallel, so only count the time of one of them $T_{\infty} = O(n/2)$.

merge subroutine is a serial while loop that iterates through all n elements, taking $\Theta(n)$ time

$$T_{\infty}(n) = T_{\infty}\left(\frac{n}{2}\right) + O(n)$$

Using the master's theorem, the span is $O(n)$

$$\text{Parallelism: } \frac{T_1}{T_{\infty}} = \frac{O(n \log n)}{O(n)} = \log(n)$$

The bottleneck in this is the sequential merge step ($O(n)$ span). A parallel merge could reduce span to $O(\log^2 n)$ and boost parallelism to $O(n/\log n)$

Q4. A fork-join parallel algorithm to do matrix multiplication is reported in a research paper with the following results. Find the inconsistency in these results.

4 processors take 80 seconds.

10 processors take 42 seconds.

64 processors take 10 seconds.

Hint: Use work and span law along with Brent's theorem to refute the research results:

The Work Law states that the parallel time can never be faster than the total work divided by the number of processors

$$T_p \geq \frac{T_1}{P}$$

solve for total work:

$$T_1 \leq P \times T_p$$

$$P = 4: T_1 \leq 4 \times 80 = 320 \mid T_{\infty} \leq 80$$

$$P = 10: T_1 \leq 10 \times 42 = 420 \mid T_{\infty} \leq 42$$

$$P = 64: T_1 \leq 64 \times 10 = 640 \mid T_{\infty} \leq 10$$

Because T_1 is a fixed property of the algorithm, it must satisfy all conditions. Therefore, the absolute maximum amount of work this algorithm can have is $T_1 \leq 320$ seconds

The Span Law states that a parallel program can never run faster than its critical path (the span), even with an infinite number of processors:

$$T_p \geq T_\infty$$

The 64-processor run finished in 10 seconds. Therefore, the span cannot possibly be longer than 10 seconds.

$$T_\infty \leq 10 \text{ Seconds}$$

Brent's Theorem:

$$T_p \leq \frac{T_1 - T_\infty}{P} + T_\infty$$

Plug in the data from the P=10 run and the maximum possible work to see what the span must be:

$$42 \leq \frac{320-10}{10} + 10 = 42 \leq 41 \text{ <- Which is FALSE}$$

The researcher said that the program should take 42 seconds with 10 processors, but Brent's Theorem proves it should take at most 41 seconds. Since 42 seconds exceeds this theoretical maximum, the 10 processors' data is incorrect.